

MPTP 0.1 - System Description

Josef Urban¹

*Dept. of Theoretical Computer Science
Charles University
Malostranské nám. 25, Praha, Czech Republic*

Abstract

MPTP (Mizar Problems for Theorem Proving) is a system for translating the Mizar Mathematical Library (MML) into untyped first order format suitable for automated theorem provers, allowing generating theorem proving problems corresponding to MML. The first version generates about 30000 problems from complete proofs of Mizar theorems, and about 630000 problems from the simple (one-step) justifications done by the Mizar checker. We describe the design and structure of the system, some limitations, and planned future extensions.

1 Availability

Mizar problems for theorem proving (MPTP) is available online at <http://alioth.uwb.edu.pl/twiki/bin/view/Mizar/MpTP>, the main packed distribution has about 70 MB and unpacks to about 100MB. It is possible to download only the basic distribution (about 300 kB) without libraries, and build the main (possibly customized) libraries from the Mizar system. MPTP should run on all sorts of Unix-like systems, where Perl and Berkeley DB are installed. You may additionally need to install the standard Perl DB_File module, providing Berkeley DB interface, which is not always included with Perl distributions. For creation of customized libraries, Mizar Linux distribution (www.mizar.org) is needed, and it is limited to Linux x86 architectures.

2 Motivation

We want to have a system that would allow closer cooperation between theorem provers and large mathematical formalization projects. It seems, that the Mizar [Rudnický 92] Mathematical Library (MML), containing nearly 800 formalized articles with the total of about 60 MB of mathematical theories, is

¹ Email: urban@kti.ms.mff.cuni.cz

currently the largest and most mathematically oriented corpus of formalized mathematics, available for this task.

This should allow things like consistent domain-based optimization and training of theorem provers, learning of lemma conjecturing, introduction of definitions and their unfolding, and provide incentive for dealing with more “real” mathematics in theorem provers e.g. by implementing efficient type handling algorithms, or integrating some efficient decision or evaluation procedures, learning how to choose premises for solving given problem from a vast repository of available assertions, or doing theorem discovery in advanced mathematical theories. Having such optimized systems could in turn further boost the feasibility and popularity of the formalization efforts, with the hope of making computers more helpful in the science that gave birth to them.

3 Description

3.1 Overview of MPTP

MPTP 0.1 at the time of writing this description consists of the following parts:

- the main Mizar-to-ATP translation tool (`fo_tool`)
- Makefiles and some very simple scripts creating the translated library from `fo_tool`’s output
- the translated Mizar library, accessible both as Prolog files and as Berkeley DB files
- Perl scripts accessing the library as Berkeley DB files, generating proof problems and providing other important functionality, like signature filtering or results parsing
- the generated proof problems
- additionally, SQL (MySQL) database of results with web interface is used for collecting and analysis of provers’ results, however this is not a part of the system distribution

The system is designed so that the transition between the Mizar format of problems and generated ATP problems is as simple, transparent, and fast, as possible. This is necessary, because the translation of the more complex Mizar logic into untyped first order format is nontrivial, and requires a lot of feedback from provers run on the translated problems (see section 3.5), and also because MML (and also Mizar) is very much “living” project, growing with new articles and changing almost every week. We also want to keep the problem generating scripts simple and documented, so that it is easy for others to experiment with creating their own customized proof problems from the library.

3.2 *fo_tool*

`fo_tool` is a standalone program, based on the Mizar implementation (written in objective Pascal). Since the source code of the Mizar checker and related utilities is only available to members of the Association of the Mizar Users, we only distribute a Linux binary, executable on x86 architectures. It is not needed, unless you want to generate your own customized library. `fo_tool` takes a Mizar article as input, and produces several files containing the translated information about various Mizar constructors, theorems, definitions and clusters exported from the article (see [Urban 03] for details of the translation, which mainly consists in relativization of the Mizar types). Additionally, `fo_tool` also collects necessary information about complete proofs of all (about 30000) exported Mizar theorems. Even though we thus collect just the minimal set of references used in the Mizar proof, problems generated from these proofs can be quite hard, because of their length and some automation (e.g. type handling) in Mizar. That's why we also export the simplest Mizar single-step justifications - checker problems. There are about 630000 of them, and they should be quite easy since the strength of the Mizar checker is limited².

Direct translation into the DFG format [Hähnle et al 96] used by the SPASS prover [Weidenbach et al 99] was chosen for the first MPTP version, since we need the strongest prover for testing the translation, and it seems, that SPASS performs best on the translated problems, probably because of its autodetection of sort theories and use of semantic blocking [Ganzinger et al 97] of ill-typed inferences. The `dfg2tptp` tool (available in SPASS distribution) can be used now to translate DFG tasks to TPTP format, but it is possible, that we will make TPTP the default format in the future, or will support more than one output format.

3.3 *The Exported Library*

The library is now a collection of several files, usually containing formulas in DFG format, expressing some part of the translated MML structure. So all translated theorems are in one file, all definitions in another, etc., and these files are Prolog readable (though sometimes quite big). We keep small index file (also in Prolog format), telling for each library file *F*, and each Mizar article *A*, at which point of *F* the translated items from *A* are placed. Since most Mizar items (e.g. theorems, definitions, constructors, etc.) are already numbered by the Mizar system (e.g. `REAL_1:70` is 70th theorem in article `REAL_1`), and the naming scheme used by our translation respects this numbering (again, the naming scheme is dealt with in more detail in [Urban 03]), it is thus usually very simple and fast (constant time) to compute a position of some item in a library file.

² The probably most detailed available description of the methods used by the Mizar checker is in [Wiedijk 00]. Some specific methods are also discussed in [Naumowicz and Byliński 02].

This approach now takes care of most of the indexation problems, necessary for fast access into the library files. The memory efficiency is solved by accessing the library files as simple Berkeley DB databases of the RECNO (record number) type.

Additionally, the library contains input files for checker problems, again in a Prolog format. Because of the number of checker problems, these files can be quite large (several MB) even for a single article, and would occupy about 1 GB, if not compressed. So for space efficiency, we keep them compressed in a special directory. Decompression and cleanup of these files are handled by the problem generating scripts.

3.4 Problem Generating Scripts

Problems creation is implemented in about 2000 lines of documented Perl modules and scripts. The basic Perl module MPTPUtils.pm provides database access to the translated library, functions for creating the basic background theory for articles, based on their environment directives, and functions for problem printing.

The Perl module MPTPSgnFilter.pm is an implementation of a signature filter, based on reasonings about the Mizar checker, and used for cutting unnecessary context (background) formulas from the problems. The criteria for adding new background formulas are derived from close inspection of the Mizar checker's work with this information, which is quite a nontrivial matter. However, the number of background formulas cut in this way from the problem is usually pretty high, and it improves the prover's chances quite significantly.

The Perl script mkproblem.pl gets names of problems, or names of articles (preceded with “-t” for theorem problems and “-c” for checker problems) as input, and produces the problem files in DFG format. It takes about 8 minutes and about 500 MB on P4 to produce all theorem problems (about 30000), producing all checker problems takes about 10 GB. Because of this speed and the problem sizes, we do not distribute the generated problems now.

3.5 The Database of Results

To facilitate the analysis of the results of provers run on MPTP, experimental SQL (MySQL) database has been set up for them (see [MPTPResults] for its SQL structure). A web interface to the database allowing arbitrary SQL selects is at <http://lipa.ms.mff.cuni.cz/phpMyAdmin-2.4.0> . This is now mainly used to look for suspicious spots in the translation, e.g. by comparing the length of a Mizar proof, with the length of the proof found by a theorem prover.

We would like to encourage MPTP users to contribute their results into the database, however, it is necessary to say, that the structure of the database may still change a lot in the early versions.

3.6 First Results

Because of limited resources, the database of results now only contains results of the SPASS prover (version 2.0), run with 4 second timelimit on P4 on all theorem problems. For about one third of the problems (8727 out of 27298 tried), the proof was found within the timelimit. We also tried the E prover [Schulz 99] (version 0.7) with the same timelimit (and in the automatic mode), solving 7737 problems. These results are not in the database yet, because of problems with proof extraction.

Two more experiments with SPASS have been conducted recently. In the first one, the signature filtering was not used, yielding problems that are on average very large (570 clauses/problem). Only about 2740 problems were proved within the 4 second timelimit. In the second experiment, a cluster of P3 was used on the filtered versions, raising the timelimit to 120 seconds. The number of proved problems raised to 10810. About 630 completions were found in this experiment, see the next section for some discussion of their possible causes and planned improvements.

Because of their number, large-scale statistics is not yet available on checker problems.

4 Problems, Limitations and Future Extensions

There are now several problems and limitations when using MPTP, their up-to-date description and suggested workarounds are present in files README_MPTP.txt and MPTPFAQ.txt distributed with the system.

Some problems arise from second order features of the Mizar language, used to deal with the infinite axiomatizations used by MML (Tarski-Grothendieck set theory). They can be solved by instantiation, and we are planning to do that in future versions. Other problems are caused by various automatizations implemented by Mizar. They can be dealt with either on the level of `fo_tool`, by watching such automatizations in Mizar, or later on the level of problem generating scripts, e.g. with methods like signature filtering. The longterm solution e.g. to arithmetical evaluations, seems to be integration of such procedures directly in ATP systems.

ATP systems that want to perform well on MPTP should also implement fast type-handling procedures, otherwise the provers may spend a lot of time, just to prove, that certain terms are properly typed.

Possible MPTP extensions include even more detailed export of the proof structure, allowing creation of harder and harder problems by following the proof structure (expanding lemmas), more experiments with the translation of Mizar types, (see [Dahn 98] for some suggestions), and maybe even direct translation to CNF format, to have consistency of skolem symbols. However the main goal remains to have at least one prover optimized very much for MML tasks, so that it can be of real use to Mizar authors.

References

- [Dahn 98] Ingo Dahn. Interpretation of a Mizar-like Logic in First Order Logic. Proceedings of FTP 1998. pp. 137-151.
- [Ganzinger et al 97] H. Ganzinger, C. Meyer, C. Weidenbach: Soft Typing for Ordered Resolution. In Proc. CADE-14, pp. 321-335, Springer, 1997.
- [Hähnle et al 96] R. Hähnle, M. Kerber, and C. Weidenbach. Common Syntax of the DFGSchwerpunktprogramm Deduction. Technical Report TR 10/96, Fakultät für Informatik, Universität Karlsruhe, Karlsruhe, Germany, 1996.
- [MPTPResults] MPTPResults.sql - SQL structure of the MPTP result database, published online at <http://alioth.uwb.edu.pl/twiki/bin/view/Mizar/MpTP>.
- [Naumowicz and Byliński 02] Adam Naumowicz and Czesław Byliński, Basic Elements of Computer Algebra in MIZAR, Mechanized Mathematics and Its Applications Vol. 2(1), August 2002.
- [Rudnicki 92] Rudnicki, P., An Overview of the Mizar Project, Proceedings of the 1992 Workshop on Types for Proofs and Programs, Chalmers University of Technology, Bastad, 1992.
- [Schulz 99] Schulz S., System abstract: E 0.3. In H. Ganzinger, ed., 16th International Conference on Automated Deduction, CADE-16, Vol. 1632 of LNAI, Springer, pp. 297-301.
- [Suttner and Sutcliffe 98] C. Suttner and G. Sutcliffe. The TPTP problem library (TPTP v2.2.0). Technical Report 9704, Department of Computer Science, James Cook University, Townsville, Australia, 1998.
- [Urban 03] Josef Urban. Translating Mizar for First Order Theorem Provers. In Andrea Asperti, Bruno Buchberger, James Davenport (eds.), Mathematical Knowledge Management, Proceedings of MKM 2003, LNCS 2594, 2003.
- [Weidenbach et al 99] Weidenbach C., Afshordel B., Brahm U., Cohrs C., Engel T., Keen R., Theobalt C. and Topic D., System description: Spass version 1.0.0, in H. Ganzinger, ed., '16th International Conference on Automated Deduction, CADE-16', Vol. 1632 of LNAI, Springer, pp 314-318
- [Wiedijk 00] Freek Wiedijk. CHECKER - at <http://www.cs.kun.nl/~frees/mizar/by.dvi>